GATE24

ISSI Project

Group Report

Group Members:

Leo, Chung Lam Shing (UID: 3036585989)

Hisankusuma Jannatan (UID: 3036579019)

Adam, Hui Sum Ho (UID: 3036584404)

Matthew, Wong Kwan Ho (UID: 3036582808)

School of Innovation, The University of Hong Kong

Runway 1331

EXECUTIVE SUMMARY

Runway 1331 is one of the largest youth hostels in Hong Kong, but its single reception desk is only staffed from 09:00 to 22:00. Guests who arrive late typically independent travellers booking through OTAs, and crowds leaving Kai Tak concerts and events can be left locked out at the door, because both identity verification and the metal-key handover still require a staff member. Extending reception to 24 hours with extra staff is the obvious fix, but at Hong Kong labour costs it is not affordable. GATE 24 automates the keycard handover instead.

Our final prototype is a minimalist, distributed kiosk paired with a pre-arrival web app. Guests complete their details online before they arrive; at the hostel they scan the QR code shown on their phone at the kiosk, the kiosk dispenses a blank RFID card and encodes it on the spot, and the guest taps straight into their room. The whole device is built from roughly forty 3D-printed parts and engraved acrylic around a Raspberry Pi 5, and costs about HK$2,000 to build, roughly forty-seven hours of minimum-wage staff time, and several times cheaper than the commercial kiosks on the market.

The one-line pitch,

Let's keep Runway 1331 open all night without the all-night payroll.

1. The visit

When and where
We visited Runway 1331 on 1 June 2026. Runway 1331 is a youth hostel and creativity / start-up hub built into the former Kai Tak airport site in Kowloon, with around 600 rooms spread across multiple blocks over a very large footprint. We spent our time at the main reception / front-desk area during operating hours, and we confirmed first-hand that reception is staffed only 09:00–22:00, not 24 hours.

What we observed
We followed the complete guest journey at the front desk and watched the receptionist work through it repeatedly. The current check-in flow looks self-service on the surface but is not: the guest scans a code and enters their booking and ID details on the property-management system (PMS), takes a selfie, and photographs their travel document, but then a paper registration card has to be printed and signed by hand, a physical metal key is issued and assigned by room type, and late-checkout is handled manually. Every one of those steps still needs a person standing at the one desk.

We also noted the wider context that shapes the problem: more than 50% of bookings arrive through OTAs such as Trip.com, so the typical guest is an outside traveller with no prior contact with staff; guest rooms have no telephones (guests reach staff only through WhatsApp / WeChat); and the direct booking site accepts only Visa and Mastercard. Individually these are small details. Together they describe a large, modern-facing operation still running on a number of manual, single-point processes.

Who we talked to
Our primary informant was a front-desk receptionist and operations associate, who walked us through the daily reception workflow, demonstrated the PMS, and spoke candidly about the practical headaches behind each step. Lost spare keys, the queue that forms at peak arrival times, and the pressure around late arrivals. Because this person performs check-in dozens of times a day and personally absorbs the friction whenever the process falls short, their account gave us something more valuable than a description of how things are supposed to work: a clear view of where the process breaks, how often, and at what cost. Much of the pain-point analysis that follows is grounded directly in this interview, supplemented by what we saw at the desk.



Figure 1. What we saw at Runway 1331 reception.

The pain points we took away

Five observations stood out, and all of them sit on the same check-in / check-out journey, because one time-limited desk has to serve thousands of rooms across multiple blocks:

EC1. No 24-hour check-in: For roughly eleven hours a day there is simply no way to check in. Late arrivals (routine given flight times and Kai Tak events) have no one to receive them, and the remote location makes a late lockout far worse than at a city hotel.
EC2. Costly staff overtime: Serving late guests means paying staff to stay; at Hong Kong's statutory minimum wage of about HK$42.1/hour, a night of overtime can cost more than the room earns (a room is roughly HK$200/night).
EC3. A single-desk bottleneck: One receptionist verifies, files, and hands keys to clusters of tired guests one by one, so queues build and the staff member becomes a single point of failure for the whole site.
EC4. Paper forms and metal keys: Signing and filing a paper registration card and hunting for the right metal key is slow, and lost spare keys are a recurring headache.
EC5. Identity verification is human-only: In Hong Kong the identity check still depends on a person comparing the guest to their document which is exactly what stops a simple self-service machine from working today.

Evidence from the field

On Rednote (Xiaohongshu) we found Runway 1331 guests publicly asking about late shuttle buses and late arrivals after March concerts, real travellers, in their own words, describing the exact moment GATE 24 is built for.

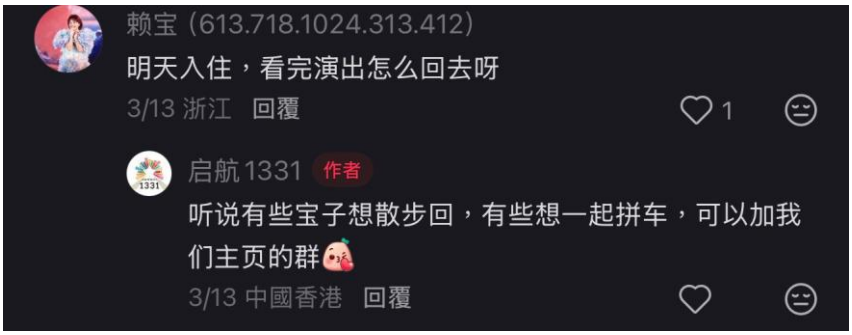


Figure 2. What we saw at Runway 1331 XHS account.

2. Problem Statement

The final problem statement

"How might we automate the keycard handover process so that travellers can independently access their rooms outside of reception hours, efficiently?"

Underneath that "how might we" framing sits the precise definition we settled on: independent travellers arriving at Runway 1331 often late at night after flights or Kai Tak events cannot

obtain verified access to their rooms outside reception's staffed hours, because both the identity check and the key handover require a staff member at the single central desk. As a result, guests are delayed or locked out at their very first impression of the property.

Root cause

The check-in process is structurally chained to staff presence. Confirming "this is the right guest, give them their room" can currently only be done by a person, at one place, within fixed hours because the existing self check-in is only partial and cannot verify identity on its own, and rooms still use physical metal keys.

Who it affects most

- "Kengie", the late-arriving guest, an OTA traveller, tired, with luggage, in an unfamiliar and remote location, whose goal is simply to get into the room quickly and safely.
- "Amy", the front-desk associate who must manually verify every guest and hand over keys, and so becomes the bottleneck and the single point of failure.

How the problem statement evolved

Our definition did not start this sharp. It narrowed in three deliberate steps, and the refinement is what kept the project tractable.

Stage	What we framed	What drove the change
Initial	A broad ambition: an automated check-in / check-out device that also handles Q&A, runs 24/7, and integrates with the PMS.	Too broad and solution-led, it described a machine, not a need, and tried to solve everything at once.
Refined	Reframed solution-agnostically around the real moment of failure: getting verified room access at arrival, outside staffed hours.	Coaching on writing human-centred, solution-agnostic statements; the visit evidence pointed clearly at the late-arrival lockout.
Final	Tightened to the single most automatable, highest-value step: the keycard handover, done independently and efficiently.	Identity verification in HK still needs a human today, so we scoped the problem to what a machine can own now, the handover and left verification to a clear future path.

Figure 3. Evolution of the problem statement,
from a broad device idea to a focused, solution-agnostic need.

That last move matters. By naming the keycard handover specifically, the statement stays broad enough to invite many designs (a full kiosk, a phone-only flow, a hybrid) yet narrow enough to be solved for one property and one decisive moment — and every clause is grounded in something we saw on the visit rather than assumed.

3. Design and Iterations

3.1 The final proposed solution

GATE 24 is a low-cost self check-in kiosk paired with a pre-departure / pre-arrival web app. The web app does the slow part (data entry and e-signature) before the guest arrives; the kiosk

does the physical part (issuing and encoding a room card) the moment they get there. Because the heavy lifting is on the guest's own phone, the kiosk itself can be small, cheap, and deployed in many places across the site rather than concentrated at one expensive desk.

The guest journey, end to end

- Book & scan QR | on phone or web, no app to install; the guest fills in their details and e-signs.
- Show QR to the kiosk | the kiosk camera reads the booking QR code.
- Card dispensed | the Raspberry Pi drives a motor to push out a blank card, then the RFID writer encodes it.
- Tap and go | the guest proceeds to the room and taps the card on the digital lock.

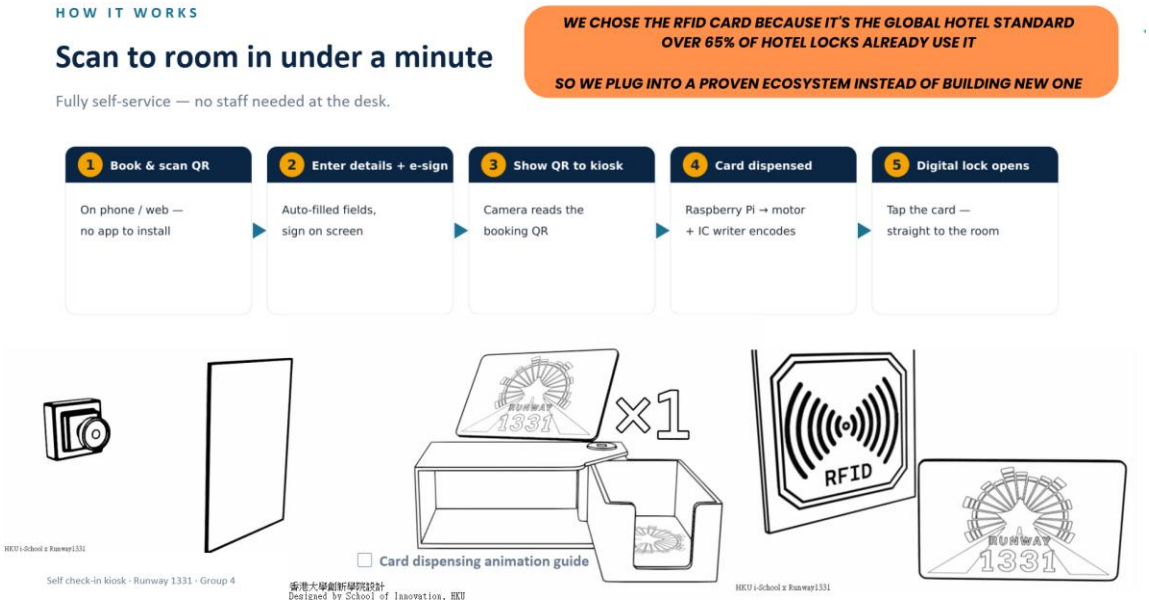


Figure 4. GATE 24 | scan to room in under a minute, fully self-service.

We chose an RFID card as the output deliberately: it is the global hotel-lock standard (well over half of hotel locks already use it in the world), so GATE 24 plugs into a proven ecosystem instead of forcing the hostel to build a new one.

3.2 How we got here (three generations of design)

The final design is the third iteration. Each generation taught us something that forced the next.

Generation 1 (the full all in one kiosk)



Figure 5. Concept sketches for first generations.

Straight after the visit, our initial proposal was a fully functional commercial-style kiosk that did everything at the desk (verify, pay, sign, issue) in one standing unit. It was the obvious answer, but it had two fatal flaws for this customer. It was expensive, and it did not actually fit a budget youth hostel's needs. Youth hostels compete on price; a HK$20,000 class terminal per location is exactly the cost structure they cannot carry.

Generation 2 (phone only check-in)

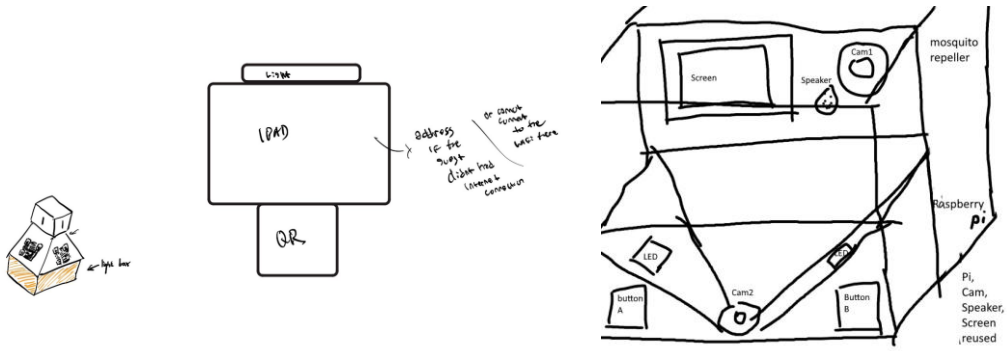


Figure 6. Concept sketches for second generations

So we swung the other way, drop the kiosk almost entirely and do everything through an online check-in flow driven by a QR code much like the mini-programs common in mainland China with at most an optional iPad or tablet to display information and open our website. It was cheap, but it hit two walls. First, a phone-only flow can really only drive PIN-code door locks, so it could not be deployed in the majority of hostel rooms, which use card locks. Second, the identity problem: in Hong Kong a person still has to verify the guest, so a purely online flow could not stand on its own.

In the beginning, we were considering making it like a usual self service kiosk. And we tried to make a difference, we wanted to make our product as useful as those in the market and also much cheaper than them, which would allow wide distribution of check-in in Runway1331 and also enhance the effectiveness.

We have considered of combining mosquito repeller to the machine as an extra function and a uniqueness. But they look unrelated and unnecessary, so this was rejected.

Generation 3 (the minimalist GATE 24 kiosk (final))

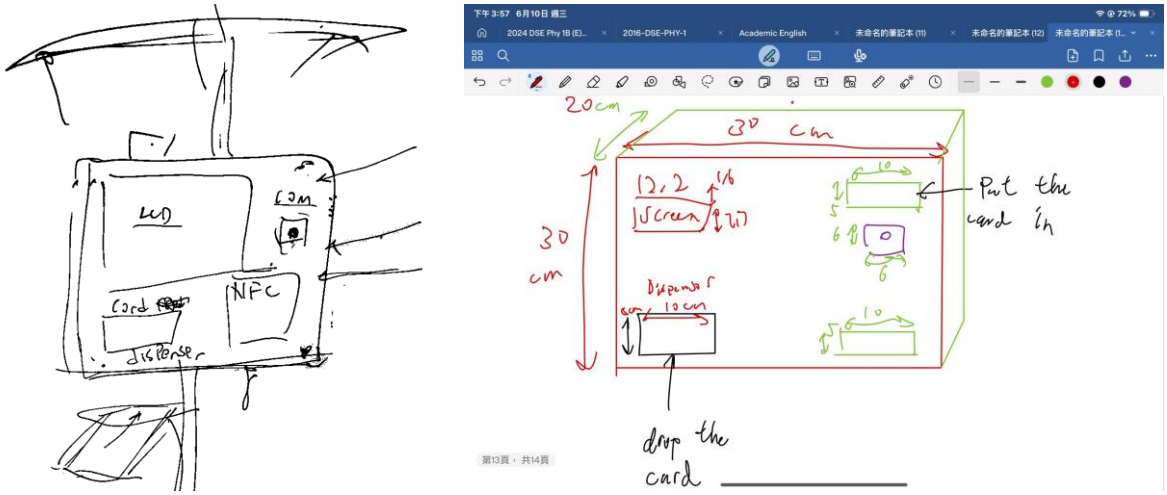


Figure 7. Concept sketches for final generations

The third generation is the win-win. We decided to replace some functions of the kiosk and make it a key dispenser. Therefore, the load for identification will be shifted to the guests' phones. By making the identification function on a web app, we also make it easy for future changes, such as PIN key or face verification.

We kept the cheap, phone-based pre-arrival website and added a deliberately minimalist kiosk whose only job is the physical keycard handover. The guest fills everything in online first; when they arrive they simply come to the kiosk, get their room number, and activate their card. This is more reliable than Gen 2 because it works with standard RFID card locks, and it hits our pain point directly while staying far cheaper than Gen 1. It is fully working today. The pre-arrival site, QR check-in, dispensing and card encoding all run smoothly in single kiosk.

Generation, What it was, Why it changed

Gen 1, Full all-in-one commercial kiosk (initial proposal)., Too costly; didn't fit a budget youth hostel that competes on price.

Gen 2, Phone-only QR check-in + optional tablet, Only drives PIN-code locks (not applicable in most hostel); identity still needs a human in HK.

Gen 3, Pre-arrival web app + minimalist card-handover kiosk., Final. Works with standard RFID locks, hits the pain point, and stays cheap.

Iteration Log

Iteration	Key Changes	Outcome/Comments
Iteration 1 (6/10)	Adopted the kiosk like design blueprint. Detached Raspberry Pi 5 from Project Alfred group *Insight.*	The direction of CAD and coding for GATE24 is determined. Linux successfully recognized camera, microphone.
Iteration 2 (6/11)	Created the VLC video-based UI.	The foundation base GATE24 is created and capable for looping animations.
Iteration 3 (6/12)	Created guest database system.	The secure and private model of GATE24 is now set.
Iteration 4 (6/13)	Initial launch of the data-pre-entry web app	Guest can enter their information in advance and generate QR code for convenience.
Iteration 5 (6/14)	Closed the main code loop.	GATE24 can successfully simulate the procedure for first time.
Iteration 6 (6/15)	Rendered animations. Received 3D model of case.	GATE24 now have an easy-to-understand guide.
Iteration 7 (6/16)	Test of RFID reader Added combability for QR code from web app.	Unfortunately, we were unable to get the RFID reader to work. The web app is now an integral part of GATE24 procedure.
Iteration 8 (6/17)	Installation of system into case. Tested servo.	All physical parts of GATE24 are now contained in a box. The dispenser of GATE24 now turning, but only 45 degrees due limited opening of acrylic cover.
Iteration 9 (6/18)	Installation of new acrylic cover with case adjustments.	GATE24 now features an opening for check-out by returning card.
Iteration 10 (6/20)	Acquired alternative RFID module.	Low level codes in GATE24 in attempt to communicate with RFID reader is now unnecessary.
Iteration 11 (6/21)	Finalize all animations.	GATE24 now features animations that more accurately reflect the procedure with warning sound.
Iteration 12 (6/22)	Installation of new RFID module. Test installation to stand.	GATE24 now have all parts implemented and working. GATE24 successfully attached to a stand and prepared as kiosk for

Iteration 13 (6/23)	Final adjustments and tests. Added demo codes to database.	The final iteration, fixing all major bugs. We decided not to fix bugs that do not affect core functionality.
Iteration 14 (6/24)	Final installation to stand. Handed out demo QR codes.	GATE24 is ready for pitch.

3.3 How the final solution works (technical detail)

Physical build

The prototype is assembled from more than forty 3D-printed PLA parts forming the frame, with a 3 mm engraved acrylic front and back case and a 5 mm engraved acrylic sign. It mounts on an adjustable hollow-aluminium standing bracket so a single design can be deployed anywhere across the hostel.

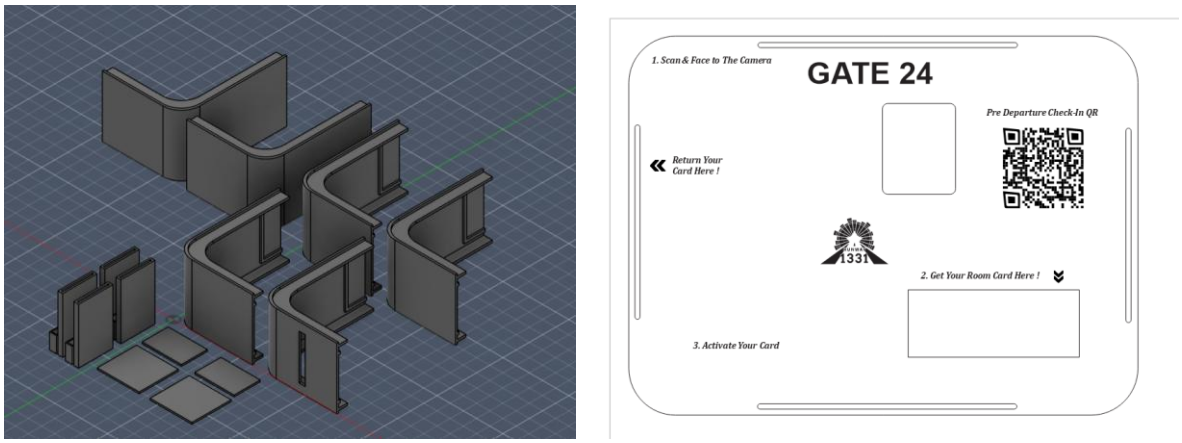


Figure 8. Sample of 3D design and CorelDraw sketch for Gate 24

Key components & wiring diagram

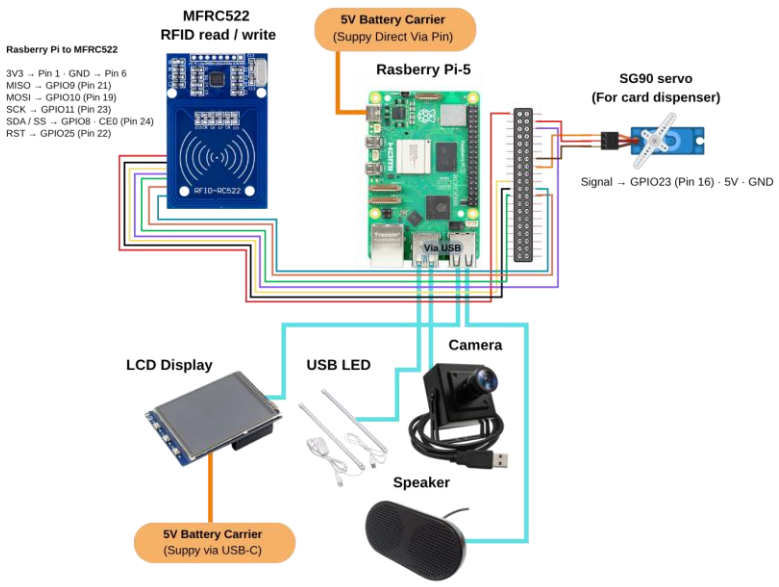


Figure 9. System Wiring diagram.

- Main frame (40 × 30 × 12 cm).
- Standing pole (190 cm tall, adjustable height)
- Raspberry Pi 5 + battery carrier (the brain of the kiosk; the battery carrier means it keeps working through a power cut)
- 4-inch touchscreen LCD (drives the user interface and the animated step-by-step guide)
- Camera (scans the booking QR code and takes the guest's photo).
- Card-dispensing system (a servo-driven mechanism that pushes out one blank card at a time)
- RFID read/write module (wipes and encodes the RFID card for the guest (MFRC522))
  Built-in LED lamp (keeps the kiosk usable at night and lights the photo capture)
- Card-return box (collects cards on check-out / return)

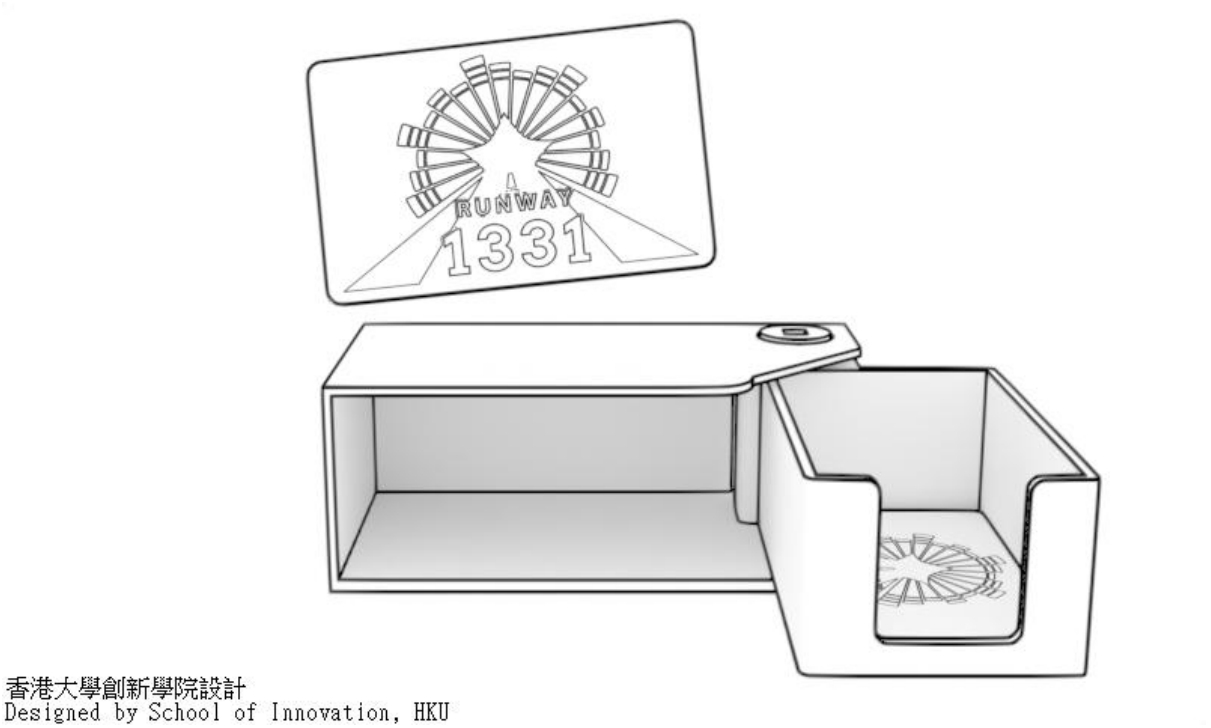


Figure 10. Sample animation for final generations

How the pieces fit together (system architecture)

A guest web app handles booking, data entry and e-signature with no install. A database does booking lookup, verification and room status. The Raspberry Pi is the hub: it reads the camera, drives the screen, fires the dispenser motor, and talks to the RFID writer that encodes the card and (in the planned final step) opens the digital lock.

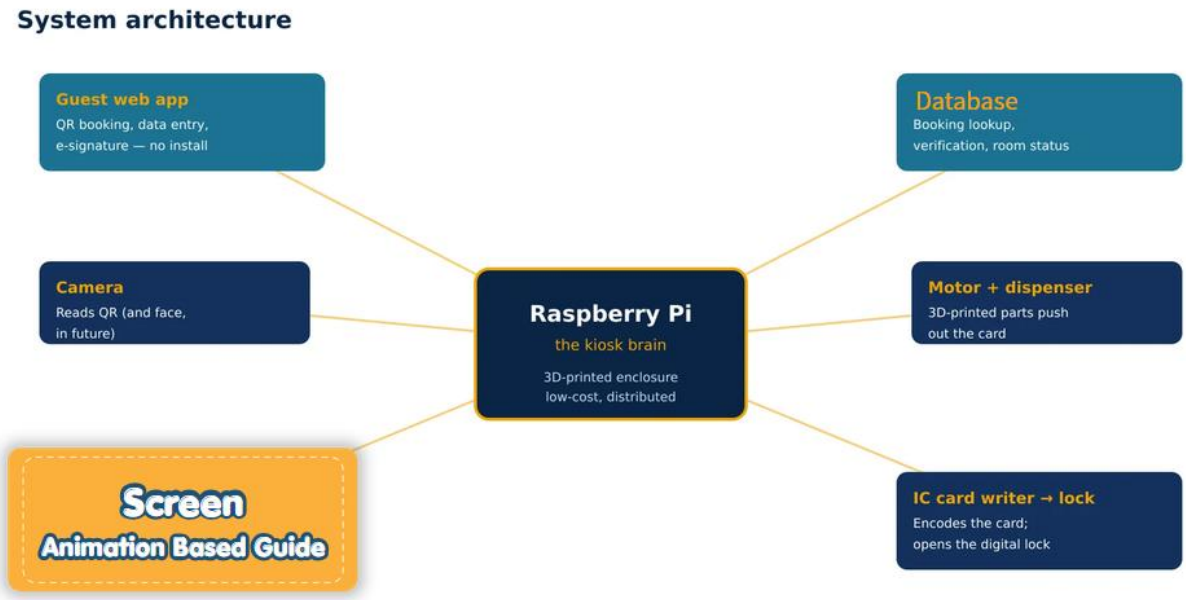


Figure 11. System architecture.

How the software works

Kiosk

The kiosk software is written in Python and built around three pieces: a Flask web service for registration, a keycard manager that holds the booking logic and database, and a kiosk controller that runs the on-screen experience and drives the hardware.

1. Privacy by design
   guest IDs are hashed, never stored as plain text. When a booking is registered, the guest's identifier is immediately converted to a SHA-256 hash, and only the hash is written to the database. There is no plain-text personal identifier sitting in any server as a deliberate “private by design” choice.

keycard.py (hashing the guest identifier)

```
@staticmethod
def generate_hash(guest_id_string):
    """Generate a SHA-256 hash for a guest ID."""
    return hashlib.sha256(
        guest_id_string.encode('utf-8')
    ).hexdigest()
```

2. Verification with a check-out cut-off. The kiosk verifies the scanned guest against the database and refuses access after 2:00 PM on the check-out date, so an expired booking cannot dispense a card.

keycard.py (verify_guest())

```
def verify_guest(self, hashed_id):
    data = self._load_data()
    if hashed_id not in data:
        return False, "Guest not found."
    guest = data[hashed_id]
    checkout = datetime.strptime(
        guest["CheckoutDate"], "%Y-%m-%d")
    cutoff = checkout.replace(hour=14)   # 2:00 PM
    if datetime.now() > cutoff:
        return False, "Access denied. Past 2:00 PM."
    return True, "Guest verified."
```

3. A safe two-phase “dispense, then activate” handover. This is the heart of the design. Issuing a card is split into two half-steps tracked as 0.5 increments: the motor first dispenses a blank card (+0.5), and only once the guest taps it on the writer is it activated with the room data and counted as a whole issued card (+0.5). A new card cannot be dispensed while one is still waiting to be activated, and the count can never exceed the booking's card limit. This prevents double-dispensing and over-issuing.

main.py (the card handover loop (simplified))

```
# main.py — dispense then write the card
servo.value = 0.9            # push the card out
system.dispense_card(guest_hash)   # +0.5 (dispensed)
...
card_id, _ = reader.write(room_str)  # encode RFID
if card_id not in cards:
    system.activate_card(guest_hash, card_id)  # +0.5
```

4. A simple admin web app generates the booking QR. A password-protected Flask page lets staff register a guest (room, card limit, check-out date) and instantly produces the QR code the guest scans at the kiosk. The guest-facing pre-arrival site is the live web demo linked on the cover.

server.py — register a guest, return a QR

```
@app.route('/add_guest', methods=['POST'])
def add_guest():
    data = request.json
    if data.get('password') != API_PASSWORD:
        return jsonify({"message":"Unauthorized"}), 401
    hashed_id = hashlib.sha256(
        data['guest_id'].encode()).hexdigest()
    # append [hash, room, max_cards, 0.0, '', checkout]
    ...   # then the page renders a QR of the guest id
```

Online Pre Check-in Website

Introduction of the Pre-Check-in Website (link and Github link)

Demo Link: https://ischool-app-psi.vercel.app/
Github Link(reference): https://github.com/matthew6218/ischool-app
(*Please click in the Github link if code reference is needed)

The Pre-Check-in Website is currently hosted on Vercel and is fully functional. It integrates smoothly into our product ecosystem and can operate normally once GATE24 is implemented in the future. The only differences from the finalized production version are the backend database connection and the incorporation of Runway1331 specific information and configurations; once these are linked, the website will serve as the complete, production-ready solution.

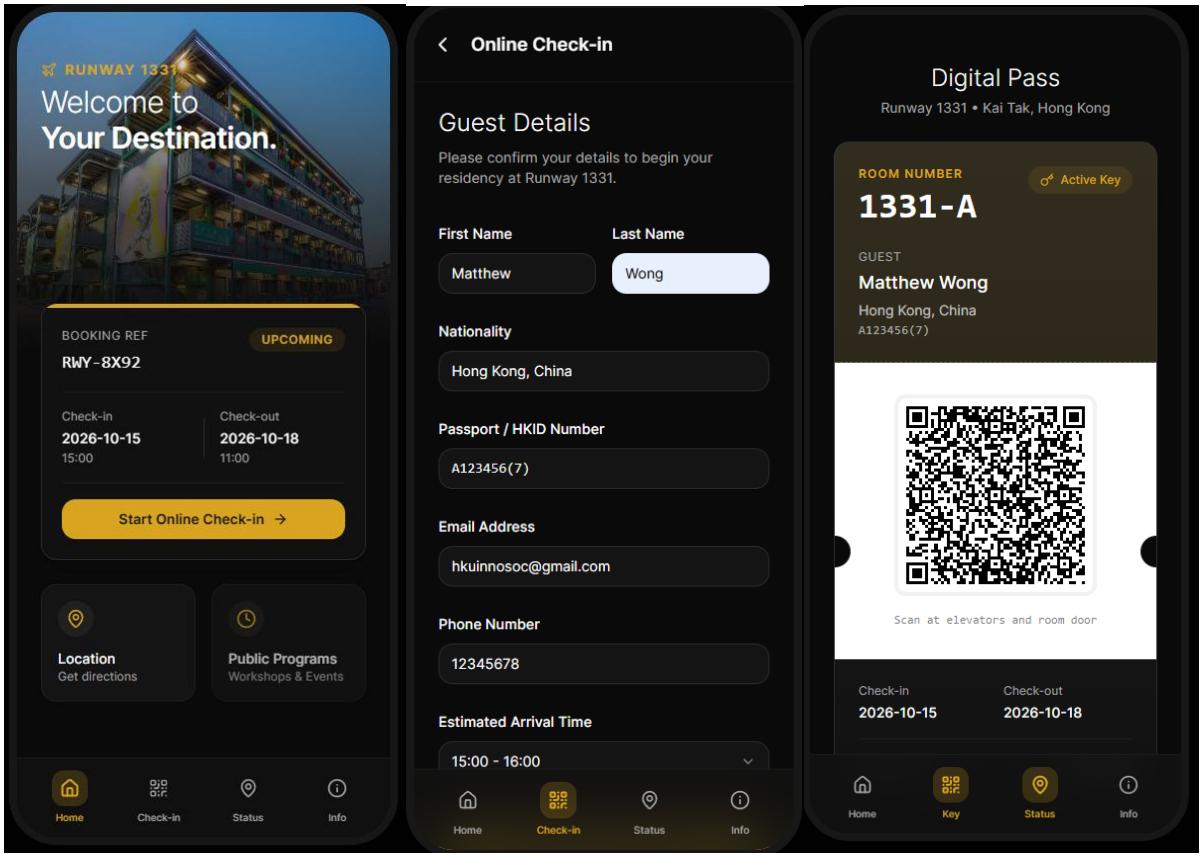


Figure 12. The guest fills in details on their phone, the kiosk guides them with on-screen animations.

Role of the Pre-Check-in Website in GATE24

The Pre-Check-in Website is the guest-facing digital layer that makes the entire GATE 24 kiosk system practical and delightful. While the Raspberry Pi kiosk handles the physical RFID card dispensing and encoding, the website handles everything that would otherwise require staff time or create queues: data entry, identity preparation, and issuing a scannable digital pass.

By completing this step on their own phone before arrival, guests who would have been locked out after 22:00 (or stuck in long queues during peak times) can now walk straight to any

distributed kiosk, scan once, receive an encoded card in seconds, and tap into their room fully independently.

The Complete 5-Step Integrated Guest Journey

(This is the flow that replaces the manual receptionist process observed during the site visit.)

1. Pre-Arrival Complete Details on Phone (Website)
- Guest receives a link (or QR) from their booking confirmation (Trip.com, direct site, etc.).
- Opens the mobile-first website (no app install needed) and fills in personal details, nationality, ID/passport info, contact, and estimated arrival time.
- Form uses real-time validation. Data is saved automatically via persistent Zustand store + localStorage.
- Website output: A beautiful digital residency pass (boarding-pass style) with embedded QR code, room assignment preview, and trip timeline.

2. Arrival at Runway 1331 (any time, even 1 a.m.)
- Guest walks to the nearest GATE24 kiosk (multiple units can be placed across the large campus because each costs only ~HK$2,000).
- No queue at a single desk. No need to wait for staff.

3. Scan QR at Kiosk (Camera + Verification)
- Guest shows the QR code from their digital pass (or booking QR) to the kiosk's camera.
- Raspberry Pi reads the QR, looks up the hashed guest ID in the local database, and verifies the booking is still valid (cut-off after 2:00 PM on check-out date).
- Kiosk output: On-screen confirmation + animated guidance.

4. Card Dispensed & Encoded (Physical Handover)
- Servo-driven mechanism pushes out one blank RFID card.
- MFRC522 module immediately encodes the card with the correct room data.
- Safe two-phase logic (dispense 0.5 + activate 0.5) prevents double-issuing or over-issuing.
- Total time at kiosk: Under 60 seconds.

5. Tap & Go Direct Room Access
- Guest takes the freshly encoded RFID card and taps it on their room's digital lock.
- No metal key hunting, no paper form signing, no staff interaction required.

Result: A tired guest who would previously have been locked out now has a smooth, premium first impression of Runway 1331 at any hour, with zero extra payroll cost.

What the Pre-Check-in Website Specifically Delivers

Component	What It Does for the Guest & the Project
Check-in Form (CheckIn.tsx)	Captures all required guest data + e-signature equivalent on the guest's own device. Real-time validation prevents errors that would otherwise require staff correction at the desk.
Digital Residency Pass + QR (TripStatus.tsx)	Generates a scannable, trustworthy digital key (QR + room preview + timeline). This QR is the exact token the kiosk camera reads to trigger secure card issuance.

Persistent State (Zustand + localStorage)	Guests can start the form on the plane, continue in the taxi, or review their pass later. Critical for travellers with intermittent connectivity.
Mobile-Native UI (MobileLayout.tsx)	Feels like a native app without installation. Works perfectly on any smartphone essential for international OTA guests.

Key Benefits for Runway 1331

- 24/7 independent check-in: Solves EC1 (no 24-hour check-in) and EC2 (costly overtime) without hiring night staff.
- Eliminates single-desk bottleneck: Multiple cheap kiosks can be placed across the large site (addresses EC3).
- Faster, paperless process: Replaces slow paper registration cards and metal key hunting (EC4).
- Better guest experience: Tired travellers complete the slow part in advance and arrive to a calm, self-service moment.
- Lowest lifetime cost: ~HK$2,000 build cost per kiosk + free web app vs. commercial kiosks at HK$20,000+ or ongoing SaaS subscriptions.

Technical Integration

The website (React 19 + TypeScript + Vite, deployed on Vercel) is designed to integrate cleanly with the existing kiosk software stack:

- QR payload from the digital pass is readable by the kiosk camera and matches the verification logic in keycard.py (hashed guest ID + checkout cut-off).
- Future backend sync: Form submission can POST to the Flask server.py (or equivalent) to store hashed data in the shared database the Raspberry Pi reads from.
- Current prototype uses client-side mock data for fast demonstration; production version will share the same persistent booking store as the kiosk.

3.4 What we prototyped, and why

We prototyped the parts that carried the most risk, the steps a slide could not prove. The point of the prototype was to show that a cheap, distributed device really can replace the desk for the keycard handover, so we built exactly the chain that has to work end to end:

- The pre-arrival web app: to prove guests can do the slow data-entry / e-signature on their own phone with no install, taking that load off the kiosk.

- QR check-in via the camera: to prove the kiosk can recognise a booking in seconds without a keyboard or staff.

- The dispensing mechanism: to prove a low-cost servo-and-3D-printed mechanism can reliably push out one card at a time is the riskiest moving part.

- RFID encoding: to prove a blank card can be written on the spot with the room data, so it works with standard hotel locks.

- The keycard logic and hashed database: to prove issuing is safe (no double-dispense, no over-issue, no plain-text identities) and survives a power cut.

In short, we prototyped the whole critical path of a single check-in, because that is what proves the concept; the polish (lock integration, full enclosure finishing) is lower-risk and deliberately left for later.

3.5 How GATE 24 compares to the market

Self check-in is not a new idea, it is a crowded, mature market so our argument is not "we built a kiosk," but "we built the cheapest, most distributable version aimed squarely at a budget hostel." Three categories already exist, and each has a structural cost we avoid:

- Commercial full kiosks (e.g. Ariane, Agilysys; mainland units such as YunShanZhu / Chengdu Allwin) is powerful but roughly HK$20,000 per device, around US$2,900–3,000 a unit. Great for a hotel lobby, unaffordable to scatter across a hostel.

- Website / SaaS verification platforms: (e.g. Canary, Operto, Duve) is flexible but billed as an ongoing subscription, around HK$5,000/month, with Duve from about US$120/month. The cost never ends, and they still depend on the guest's own device and a compatible lock.

- A manual receptionist: is about HK$60/hour all-in, the status quo, which is exactly the cost the hostel is trying to escape after hours.

Our approach is Typical cost, trade-off for a youth hostel

Approach	Typical cost	Trade-off for a youth hostel
Commercial full kiosk	~HK$20,000 / device	Too expensive to distribute across a large, multi-block site.
Website / SaaS platform	~HK$5,000 / month	Never-ending subscription; depends on guest devices and lock type.
Manual receptionist	~HK$60 / hour	The after-hours cost the hostel wants to remove.
GATE 24	**~HK$2,000 to build**	Lowest lifetime cost; distributable, private-by-design, ~99% uptime.

Figure 13. GATE 24 against the three existing approaches.

Why ours is different

- All-round cheaper , about HK$2,000 to build equivalent to roughly forty-seven hours of minimum-wage staff time and the lowest lifetime cost of any option, with no per-month subscription.
- Distributed by design, because the kiosk is small and cheap, many can be placed across Runway 1331's large footprint, instead of one costly terminal at one desk a fit no commercial kiosk can match on cost.
- Doesn't depend on labour or on the guest's device, the card-handover works on its own hardware, giving roughly 99% uptime.
- Private by design, only SHA-256 hashes are stored no plain-text guest identities on any server, and the booking data runs locally rather than in a third-party cloud.

4. Future Improvement

4.1 Where the project is today

GATE 24 is an almost fully working kiosk prototype, past a proof of concept, and close to, and already serve as a minimum viable product.

Is it an MVP yet? Almost. The whole check-in chain works except the final lock-opening step. If we had more time, closing that one integration encoded card to live digital lock, tested over a real night of arrivals is what turns today's prototype into a deployable MVP a hostel could actually run unattended.

During the testing of GATE24, we invited different people to try our product. Our product could successfully guide people to activate their key cards every time; the interface of the web app is clear enough; the guidance from the screen is clear enough; the IC card encoding and coding part worked. Everything worked.

4.2 From kiosk to fully automatic

The reason we built a phone-first system was always to leave room to automate identity verification the one job that still needs a human today without adding hardware cost. Our roadmap closes that gap:

- At-kiosk face capture (current idea): the camera already takes a live photo linked to the booking; staff can review those photos during working hours, or at check-out, as a first identity check.
- Face recognition: real-time similarity scoring against the booking photo with a configurable threshold: privacy-first, processed locally. The same approach is trusted by banks, the HKJC and Alipay, so it is practical, and once it works the flow becomes genuinely 24/7 and unattended.
- iAM Smart integration: Hong Kong residents verify instantly with their iAM Smart (智方便) app QR for official HKID confirmation.
- Bluetooth & NFC verification: check the guest is physically at Runway 1331 and validate the travel document via NFC.
- Manual remote review: staff can remotely compare the kiosk photo with the passport and trigger a room inspection if something looks off.

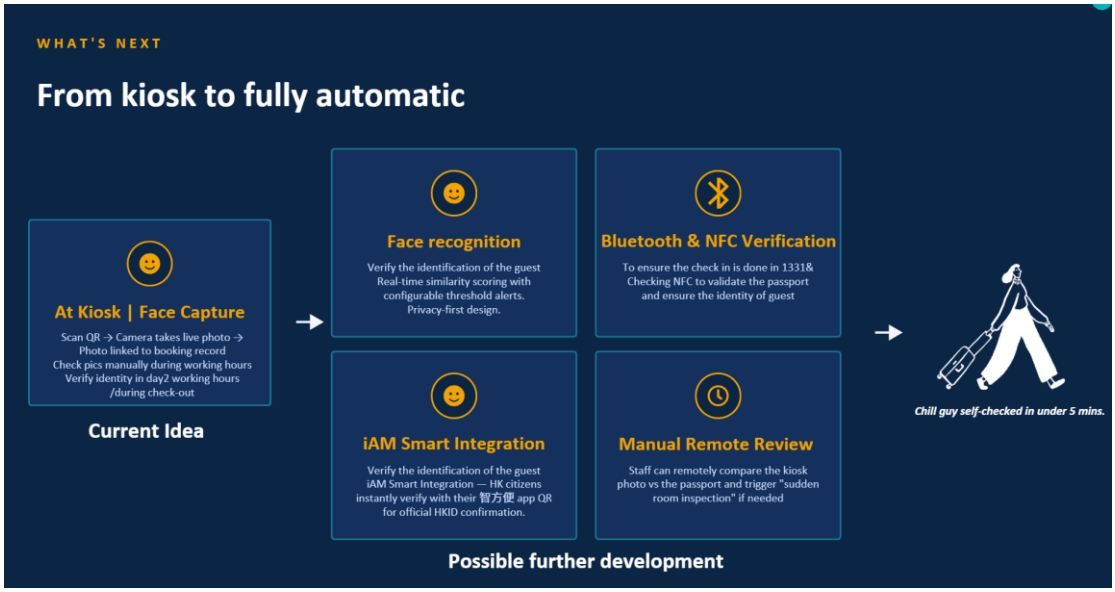


Figure 14. The path from today's kiosk to fully automatic 24/7 self check-in.

4.3 A business model that turns a cost into revenue

Every existing product on the market is a pure cost. Because GATE 24 owns the guest's arrival moment cheaply, it can also open new revenue the hostel cannot capture today so the device can pay for itself:

- Ad space: curated slots for events, partners and upsells on the kiosk and web app.
- Laundry tokens: margin on every washer credit sold.
- Room service / F&B: food sales the booking site cannot capture today.
- Upsells: upgrades, early check-in, late check-out and room selection.

With no per-block commercial terminals and no per-guest subscription, the ad, token and F&B income can offset the hardware a structural cost advantage on top of the lower build price.

5. Conclusion

We began with a tired guest standing at a locked door at 1 a.m. after a Kai Tak concert, and a hostel that could not afford to keep the desk staffed to let them in. Through three design iterations a full kiosk that was too costly, a phone-only flow that could not handle real locks or identity, and finally a hybrid that does both jobs cheaply so in the end we arrived at GATE 24, a roughly HK$2,000, distributable, private-by-design kiosk that automates the keycard handover and lets Runway 1331 stay open all night without the all-night payroll. It is not perfect but it is a working answer to a real problem we watched happen, and a credible path to a 24/7, fully self-service arrival.

6. References

Bambu Lab. (n.d.). *Bambu Studio* (Version 2.x) [Computer software]. https://github.com/bambulab/BambuStudio

NXP Semiconductors. (n.d.). *MFRC522: Standard performance MIFARE and NTAG frontend* [Data sheet / IC documentation]. https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf

Python Software Foundation. (202x). *Python* (Version 3.x) [Computer software]. https://www.python.org/

Market and product references named in the comparison (Section 3.5). Figures are drawn from publicly listed vendor pricing and industry reporting and should be re-checked against current quotes before any procurement decision.

- Canary Technologies — mobile check-in, guest verification and mobile keys (SaaS).
- Operto; Duve — contactless check-in / verification platforms (subscription; Duve from ~US$120/month).
- Ariane; Agilysys — commercial self-check-in kiosks (~HK$20,000-class hardware).
- YunShanZhu PMS (雲閑住); Chengdu Allwin Intelligent Equipment Co., Ltd. — self-service hotel terminals (~¥21,800 / ~US$2,900–3,000).
- Hong Kong statutory minimum wage — HK$42.1 / hour (used for the staffing-cost comparison).
- Field evidence — Runway 1331 guest posts on Rednote (Xiaohongshu) regarding late arrivals / shuttle timing.